B10. Reliability of Serverless Web Applications — Blue Track

Mathew Shereni CSCI 5360 Web Technologies, Fall 2026 Project Midterm Deliverable October 20, 2026

1 Assigned Project and Scope

Assigned project: B10. Reliability of Serverless Web Applications — Blue Track.

Objective. I will build a small serverless product-catalog Web API on AWS Lambda behind Amazon API Gateway, together with a controllable simulated dependency and an automated workload generator. Using this testbed, I will measure when the API violates a stated service-level objective (SLO) under cold starts, rising concurrency, slow dependencies, timeouts, and transient failures, and how much two reliability mechanisms—retries with exponential backoff and jitter, and in-function response caching (a per-instance, in-memory TTL cache held inside each Lambda execution environment)—reduce those violations, at what cost in latency, Lambda charges, and data freshness. The intended beneficiaries are developers and site-reliability engineers of small serverless APIs who must decide which mechanisms are worth their added cost and complexity.

Course topics. (i) Web APIs and HTTP semantics: status codes, timeouts, and the idempotency of `GET` versus `POST`, which determines what may safely be retried; (ii) Web caching: freshness lifetime (TTL) and staleness; (iii) cloud and serverless deployment of Web applications; (iv) Web performance measurement: latency percentiles and load generation.

Tier	Task	Measurable completion criterion
Essential	**T1. Testbed:** catalog API (`GET /products/{id}`, `POST /orders`), simulated dependency service with a configurable fault injector, deployed by infrastructure as code.	One command deploys, and one destroys, the full stack in us-east-2. Over 10,000 calls, a 10% fault setting yields 9–11% injected failures, and a +300 ms delay setting raises median dependency latency by 270–330 ms.
Essential	**T2. Reliability mechanisms:** deadline-aware retries with exponential backoff and full jitter for product reads; per-instance in-memory TTL cache in the Lambda execution environment (no external store); both switched on or off by configuration only.	All correctness tests in Section 5 pass; switching a mechanism needs no code change; every response reports attempts, cache hit, and cold-start status in headers.
Essential	**T3. Measurement pipeline and evaluation:** k6 scenarios for all conditions, run orchestrator, metric and cost calculator, analysis scripts.	The full grid (4 configurations $\times$ 7 conditions $\times$ 5 repetitions $=$ 140 runs) completes with raw per-request data saved; one script regenerates every final table and figure from the raw data.
Ambitious	**T4. Circuit breaker** with graceful fallback (stale cached or degraded response), evaluated in the outage condition (H3).	State-transition tests pass (closed $\rightarrow$ open $\rightarrow$ half-open $\rightarrow$ closed); condition C7 is run with the breaker for all five repetitions.
If Time Permits	**T5. Request deduplication:** idempotent `POST /orders` using client-supplied idempotency keys stored in DynamoDB, so retried writes return the original response without a duplicate order or stock decrement; enables retries for writes.	1,000 requests sharing one key, sent by 50 concurrent clients, produce exactly one order record and one stock decrement; every final response is 200 with an identical body hash, and all except the first carry `Idempotent-Replay: true`.

Table 1: Project tasks and completion criteria.

Exclusions. No end-user frontend; no authentication or authorization study (the API is a test service); no real payments or inventory; no multi-region failover; no platform-paid cold-start mitigations (provisioned concurrency); no shared external cache such as ElastiCache or the API Gateway REST cache (Section 6). Only one setting per mechanism is tested, so finding optimal retry or TTL parameters is out of scope.

Scope refinements. Of the mechanisms suggested in B10, caching and retries with backoff are the two studied mechanisms. Graceful fallback is part of ambitious task T4 and request deduplication is stretch task T5. Until T5 is complete, retries apply only to idempotent `GET` requests, so a retried write can never create a duplicate order. All conditions and metrics required by B10 are retained (Table 4).

2 Software Implementation Plan

Table 2 lists the software I will implement. Code drafted with AI assistance is reviewed, covered by the tests in Section 5, and must be explainable by me. Reused dependencies supply infrastructure; the logic column is my contribution.

Table 2: Software modules.

Module	Responsibility and inputs/outputs	Logic	Reused dependencies
M1. Catalog API handler	In: HTTP API event for `GET /products/{id}` or `POST /orders` (JSON `product_id`, `quantity`; `Idempotency-Key` header). Out: JSON body plus headers `X-Attempts`, `X-Cache`, `X-Cold-Start`, `X-Env-Id`, `X-Data-Version`, `X-Request-Id`.	Validate input (ID integer 1–1,000, quantity 1–10, else 400; unknown ID 404). Read mechanism flags once at initialization. Set a cold-start flag and random environment ID at module scope, so they describe the execution environment. Map outcomes to 200, 400, 404, 503 (dependency failed), or 504 (deadline exceeded). Emit one structured JSON log line per request.	Python 3.12 Lambda runtime and API Gateway HTTP API provide invocation, routing, and TLS.
M2. Reliability middleware	In: dependency call, cache key, request deadline, flags. Out: value with attempt count and cache status, or a typed failure.	*Cache:* dictionary `key`→(value, version, expiry); serve if unexpired, otherwise call through and store; `GET` only. *Retry:* retry only on 5xx, timeout, or connection error, never on 4xx; at most 4 attempts; sleep $U(0, \min(1000, 100 \cdot 2^n))$ ms; each attempt's timeout is $\min(1000\,\text{ms}, \text{time left})$; no attempt if under 50 ms remain; writes retried only with T5. *Breaker (T4):* opens after 5 consecutive failures, 15 s cooldown, one half-open probe; fallback is a stale cache entry or 503 with `X-Degraded`.	Python standard library only. No retry library is used, so every rule is explicit and testable.
M3. Dependency client	In: item ID or reservation, per-attempt timeout. Out: parsed JSON or error class (`Timeout`, `ServerError`, `ClientError`).	Create one connection pool at initialization and reuse it across warm invocations; sign requests; classify failures so M2 knows what is retryable; record per-attempt latency.	urllib3 (pooling, timeouts); botocore (SigV4 signing for the IAM-protected function URL).
M4. Simulated dependency and fault injector	In: `GET /items/{id}`, `POST /reserve`; fault config (mode, rate, delay, outage window, run start, seed). Out: item record or reservation result, or an injected 503 or delay.	Compute seconds since run start; return 503 inside the outage window; otherwise fail with the configured probability using a seeded generator; sleep the configured delay; read the item, or decrement stock only if stock $\geq$ quantity (else 409). Log every injected fault as ground truth.	Lambda function URL (HTTPS endpoint without gateway charge); boto3 and DynamoDB on-demand (storage, conditional writes).
M5. Workload generator	In: condition, configuration label, run ID, seed. Out: requests at the planned rate; one JSON record per request.	Arrival-rate executors per Table 4; Zipf (s=1) sampler over 1,000 IDs from a precomputed CDF and seeded generator; 90/10 read/write mix; unique idempotency key per order; 100 simultaneous requests for C2; 10 s client timeout; every request tagged with run ID.	k6 (scheduling, HTTP client, timing, output).
M6. Experiment orchestrator	In: experiment plan (configurations, conditions, repetitions, seed). Out: completed runs in S3 and a run manifest.	Seed the catalog; derive a randomized interleaved run order from the seed; per run, set flags and fault config on both functions and wait for the update; 30 s warm-up except C2; start k6 and the price updater (1% of prices every 10 s, with a version log); re-queue invalid runs (dropped iterations or CPU $> 70\%$); upload raw files to S3.	boto3 (Lambda configuration, S3, CloudWatch metrics).
M7. Metrics, cost, and analysis	In: k6 records, CloudWatch `REPORT` lines, fault and version logs. Out: per-run metrics, cell summaries, the tables and figures in Section 4.6.	Join client and Lambda records by request ID; classify failures; nearest-rank percentiles; 1-minute SLO windows; recovery time (5 s buckets, 30 s stability); cost C; stale reads from the version log; median and range across repetitions.	pandas and NumPy (tables), matplotlib (figures), CloudWatch Logs Insights (log queries).
M8. Infrastructure as code	In: region, memory, reserved concurrency. Out: deployed stack and API URL; one-command teardown.	Two functions, HTTP API routes, function URL, three on-demand DynamoDB tables, least-privilege IAM roles, 14-day log retention, $20 budget alarm, EC2 load host.	Terraform AWS provider.

3 Architecture and Technology Decisions

Figure 1 traces one read during condition C4 (transient faults) with Retry+Cache. k6 on the EC2 load host sends `GET /products/42` over HTTPS to the API Gateway HTTP API, which invokes the Catalog API function. The product is not in this environment's cache, so the middleware calls the dependency's function URL with a 1,000 ms timeout. The fault injector returns 503, so the middleware waits a jittered 0–100 ms and tries again; the second attempt reads item 42 from DynamoDB. The result is cached for 60 s and returned with `X-Attempts: 2` and `X-Cache: MISS`. Both functions write `REPORT` lines to CloudWatch Logs. After the run, the orchestrator uploads the k6 records to S3, and the analysis scripts join them with the logs by request ID. Configuration changes travel only on the control path, through the Lambda API, before each run.

Figure 2 shows the request pipeline inside the Catalog API function: handler → cache → breaker (T4) → retry policy → dependency client. The cache, breaker state, cold-start flag, and environment ID live at module scope, so each execution environment has its own copy; this is the design choice that H2 and H3 test. The data model has three DynamoDB tables. Product (ID, name, price, stock, version) is read by the dependency; stock never goes below zero because decrements are conditional. Order (order ID, product ID, quantity, created time) references one product. IdempotencyRecord (key, status, response hash, response body, expiry) is written with a condition that the key does not already exist, so each key yields at most

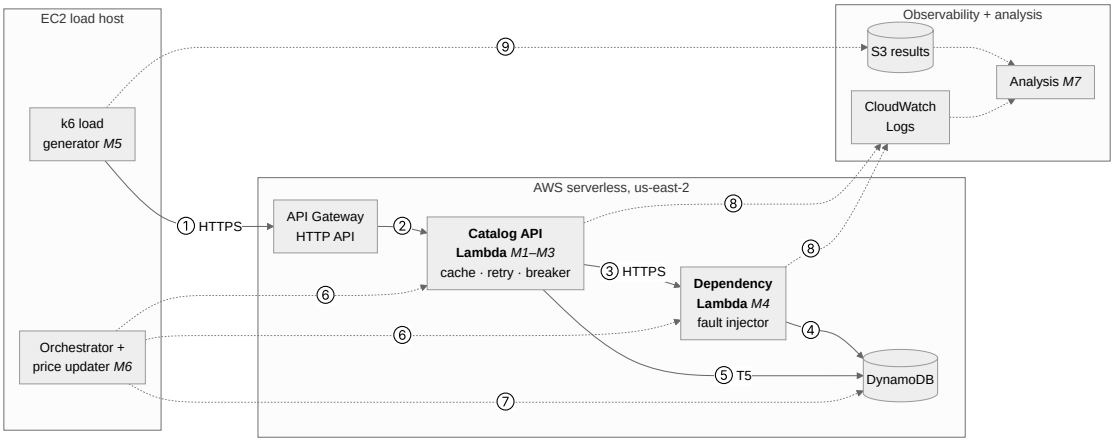
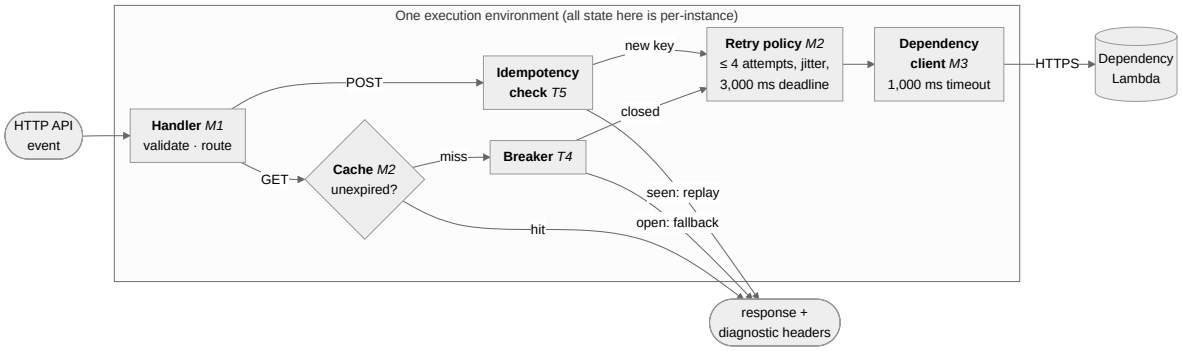


Figure 1: Architecture. Solid arrows: request path; dashed arrows: measurement and control path. ① `GET /products/{id}` and `POST /orders` at rate $\lambda(t)$; ② Lambda proxy integration; ③ SigV4-signed call to the dependency function URL, 1,000 ms per-attempt timeout; ④ `GetItem` and conditional `UpdateItem` (stock); ⑤ conditional `PutItem` of the idempotency record (T5); ⑥ `UpdateFunctionConfiguration` sets mechanism flags and the fault schedule before each run; ⑦ the price updater increments product versions; ⑧ `REPORT` and JSON log lines; ⑨ raw per-request records uploaded after each run.



(a) Request pipeline inside one execution environment

Product	
`product_id`	PK, 1–1,000
`name`, `price`	
`stock`	decrement only if $\geq$ quantity
`version`	incremented by price updater

Order	
`order_id`	PK
`product_id`	FK $\rightarrow$ Product (many-to-one)
`quantity`	1–10
`created_at`	

IdempotencyRecord (T5)	
`idem_key`	PK, put only if key not present
`status`	`IN_PROGRESS` or `COMPLETED`
`order_id`	FK $\rightarrow$ Order (at most one)
`response_hash`, `response_body`	
`expires_at`	TTL 24 h

(b) DynamoDB data model and constraints

Figure 2: Internal design of the Catalog API function and data model.

one order (T5).

Technology choices. *Lambda with API Gateway HTTP API*: B10 requires a serverless platform, and the HTTP API costs $1.00 per million requests versus $3.50 for the REST API, whose cache we do not use (Section 6). *Python 3.12*: fast cold starts and a standard library sufficient for the middleware. *DynamoDB on-demand*: scales to the 2,000 req/s ramp without capacity planning, and conditional writes enforce the stock and idempotency constraints. *k6 on a 2-vCPU EC2 instance in us-east-2*: arrival-rate executors provide the open workload, and running in the same region removes home-network noise. *CloudWatch Logs*: Lambda's `REPORT` lines provide billed duration and initialization time (cold starts) without extra instrumentation. *Terraform*: one-command deploy and teardown makes the testbed reproducible and limits idle cost. Dependencies: the $119 AWS credit and the Lambda concurrency quota of 1,000.

4 Track-Specific Design: Problem Formulation and Methodology

4.1 Research Question and Hypotheses

Primary research question. To what extent do retries with exponential backoff and jitter and in-function response caching improve the reliability of an AWS Lambda–based Web API relative to an unprotected baseline under cold-start, burst-concurrency, and dependency-failure conditions, and what trade-offs do these mechanisms introduce in latency, throughput, and cost?

Sub-questions. RQ1 (fault tolerance): how do retries and caching, alone and combined, affect p_{95}/p_{99} latency and error rate under transient faults, slowdowns, timeouts, and outages? **RQ2 (concurrency):** how does cache effectiveness (hit ratio, latency, dependency calls avoided) change as concurrency rises and cold starts create empty caches? **RQ3 (cost):** what does retry overhead cost in Lambda charges, how much do cache hits save, and what staleness does the 60 s TTL introduce?

Hypotheses.

- **H1 (Retries).** Under independent transient faults (C4), retries reduce the error rate from about 10% to below 1%, matching the analytical prediction, at about 1.11 dependency calls per request. Under sustained slowdowns (C6) and outages (C7), they push p_{99} toward the 3,000 ms deadline and raise billed duration in proportion to attempts without restoring SLO compliance.
- **H2 (Cache).** Caching reduces p_{95} read latency, billed duration, and dependency calls; because each execution environment starts with an empty cache, its hit ratio and benefit decline as arrival rate and concurrency rise (C2, C3).
- **H3 (Circuit breaker — Ambitious).** In a sustained outage (C7), an open breaker returns fallback responses at near warm-path latency instead of waiting for timeouts, and recovery is bounded by its 15 s cooldown; because breaker state is per instance, time-to-open grows with concurrency.

Why it matters. Retries and caching are usually the first reliability mechanisms a team adds, yet each can backfire: retries multiply load on a struggling dependency and add billed duration, while a per-instance cache may be nearly empty during bursts, exactly when Lambda creates many new execution environments. Measuring both effects under the same controlled conditions tells a developer which mechanism to adopt first and when it stops helping.

4.2 Problem Formulation

Each run is defined by three inputs: a *configuration* $M \in \{\mathsf{Base}, \mathsf{Retry}, \mathsf{Cache}, \mathsf{Retry{+}Cache}\}$ (plus CB in ambitious task T4); a *condition* F (Table 4) that sets the injected fault type, rate, delay, and schedule; and a *workload* W given as an open arrival rate $\lambda(t)$, a 90%/10% read/write mix, and Zipf-distributed product IDs. Everything else is fixed by Table 3. Base keeps the same 1,000 ms per-attempt timeout as the other configurations but makes a single attempt and has no cache, so differences are caused by the mechanisms rather than by the timeout.

For every request i, k6 records client-side latency L_i and HTTP status s_i, and the API returns headers reporting attempts a_i, cache hit h_i, cold start c_i, execution-environment ID, and data version. A request *fails* if $s_i \geq 500$, $s_i = 429$ (throttled), or it exceeds the 10 s client timeout. The metrics are:

- latency percentiles p_{50}, p_{95}, p_{99}; throughput (successful responses per second); error rate (failed / total);
- retry overhead $\bar{a}$ (dependency calls per request), cache hit ratio H, and cold-start count;
- stale-response rate: the fraction of cached reads whose data version is older than the latest version at serve time (a seeded updater changes 1% of prices every 10 s);
- *SLO compliance*: a 1-minute window complies if $p_{95} \leq 500$ ms and error rate $\leq 1\%$;
- *recovery time* $R = t_{\text{rec}} - t_{\text{end}}$, where t_{end} is when the injected fault stops and t_{rec} is the start of the first 5-second bucket from which every bucket for 30 s meets both SLO thresholds;
- *estimated cost* per 1,000 successful requests,

$$C = \frac{1000\left(N_{\text{gw}}\,p_{\text{gw}} + N_{\text{inv}}\,p_{\text{inv}} + G\,p_{\text{GBs}} + R\,p_R + W\,p_W\right)}{N_{\text{succ}}},$$

where N_{gw} is API Gateway requests, N_{inv} is Lambda invocations of both functions, and G is billed GB-seconds from CloudWatch `REPORT` lines, priced at us-east-2 list prices (HTTP API $1.00 per million requests; Lambda $0.20 per million invocations and $0.0000166667 per GB-second), and R, W are DynamoDB read and write request units (on-demand: $0.125

and $0.625 per million). DynamoDB is included because it is not negligible: at our 10% write mix it is roughly 10% of per-request cost, and the free tier covers only provisioned capacity, which cannot absorb the 2,000 req/s ramp. The free tier is excluded so that results generalize; CloudWatch log ingestion is excluded as measurement overhead common to all configurations.

Assumptions. Injected faults are independent per call except in the scheduled outage; the load generator is not the bottleneck, which is checked in every run (k6 reports zero dropped iterations and EC2 CPU stays below 70%).

4.3 Positioning

Brooker's analysis of exponential backoff and jitter [1] and the Amazon Builders' Library article on timeouts and retries [2] argue, from simulation and operational experience, that jitter prevents synchronized retry storms and that unbounded retries amplify overload. I test these claims end to end on a Lambda API and add a per-request cost accounting under Lambda's pay-per-duration billing. Wang et al. [3] and Shahrad et al. [4] characterize cold starts and the lifecycle of execution environments on commercial serverless platforms. I build on their observation that new environments start cold to test a consequence for application-level caching: a per-instance cache should lose effectiveness as concurrency rises (H2). The contribution is a controlled, reproducible comparison of existing mechanisms on one current platform, not a new mechanism.

4.4 Experimental Design

Comparison groups. The four configurations above. Base is the unprotected baseline required by B10, and Retry+Cache tests whether the two mechanisms interact. **What varies:** configuration and condition (Table 4). **What stays fixed:** all parameters in Table 3, the catalog, the random seed, and the client location.

Region	AWS us-east-2 (Ohio)	**Front door**	API Gateway HTTP API (no gateway cache)
Function	Python 3.12 on Lambda, 256 MB, 5 s timeout	**Load generator**	k6 on an EC2 instance in us-east-2
Per-attempt timeout	1,000 ms	**Request deadline**	3,000 ms (no new attempt that would exceed it)
Retry policy	At most 3 retries (4 attempts); exponential backoff, base 100 ms, cap 1,000 ms, full jitter	**Cache**	In memory per execution environment; `GET /products/{id}` only (never orders); TTL 60 s
SLO	$p_{95} \leq 500$ ms and error rate $\leq 1\%$ per 1-minute window	**Concurrency**	Quota 1,000 (raised from 10); reserved 400 per function as a safety cap

Table 3: Fixed experimental parameters (injected faults are in Table 4).

Condition	Injected at the dependency	Workload	B10 requirement / hypothesis
C1 Warm steady	none	50 req/s, 3 min	reference
C2 Cold start	config update forces new environments	100 simultaneous reads, then 50 req/s for 30 s	cold starts; H2
C3 Ramp	none	100, 250, 500, 1k, 2k req/s, 60 s each	increasing concurrency; H2
C4 Transient	10% of calls return 503	50 req/s, 3 min	transient failures; H1
C5 Slow	+300 ms per call (below timeout)	50 req/s, 3 min	slow dependency
C6 Timeout	+1,500 ms per call (above timeout)	50 req/s, 3 min	timeouts; H1
C7 Outage	100% failure from minute 1 to 2	50 req/s, 3 min	recovery time; H3

Table 4: Experimental conditions. Steady conditions start with a 30 s warm-up that is excluded from analysis.

Repetitions and rationale. Each of the 28 configuration–condition cells is repeated five times (140 runs). Runs are executed in randomized, interleaved order within each session, so a slow period on the shared cloud affects all configurations alike instead of biasing one. Five repetitions expose between-run variability, which in cloud measurements is usually larger than within-run variability, while keeping the total load at about 6 million requests (an estimated $15–25 of the $119 credit). The open arrival-rate workload (k6 `constant-arrival-rate` and `ramping-arrival-rate`) keeps sending requests at the planned rate even when the API slows, so slowdowns appear in the latency data instead of silently reducing load.

4.5 Data

All data are generated in my own AWS account; no third-party systems or personal data are involved, so there are no access or licensing constraints. **Catalog:** 1,000 synthetic products (ID, name, price, stock, version) generated by a seeded script and loaded into DynamoDB. **Requests:** product IDs follow a Zipf distribution ($s = 1$), so a few popular products receive most reads, as in real stores; orders use random valid IDs and unique idempotency keys. **Collected records:** per-request k6 output (latency, status, headers) and CloudWatch `REPORT` lines (billed duration, init duration, memory used). **Inclusion and exclusion:** warm-up periods are excluded; a run with dropped k6 iterations or EC2 CPU above 70% is discarded and repeated; no other data are removed. **Ground truth:** the injected fault schedule and the data-version log are used to check error attribution and staleness.

4.6 Analysis

For each run I compute the metrics above; for each cell I report the median of the five per-run values and their range (minimum–maximum). Because between-run variation dominates, a difference between configurations is reported as an effect only if their ranges do not overlap; otherwise it is reported as inconclusive. Planned outputs: (1) an *SLO verdict matrix* (configuration $\times$ condition, share of compliant windows), which answers B10's question of when the API violates its target; (2) a table of

$p_{50}/p_{95}/p_{99}$, error rate, throughput, and $\bar{a}$ for C4–C7, including measured versus analytical error rate under C4 ($0.1^4 = 0.01\%$ with four independent attempts) (RQ1, H1); (3) a time series of error rate and p_{99} during the C7 outage for each configuration (recovery time; H3 for ambitious task T4); (4) a figure of hit ratio, p_{95}, cold starts, and concurrent executions versus arrival rate in C3 (RQ2, H2); (5) a cost table: cost per 1,000 successful requests and stale-response rate per configuration (RQ3). Finally, I rank the mechanisms by the reduction in non-compliant SLO windows relative to added cost and added complexity (lines of code and number of tunable parameters).

4.7 Limitations

- **Cloud variability.** Shared infrastructure makes latency drift over time. Mitigation: randomized interleaved run order, five repetitions, and reporting ranges instead of single values.
- **Synthetic dependency and faults.** Real dependencies fail in correlated and partial ways. Mitigation: both independent faults (C4) and a correlated outage (C7) are tested, and conclusions are limited to the fault models tested.
- **Single platform and settings.** One region, runtime (Python 3.12), memory size, retry policy, and TTL are used, so the study cannot establish optimal parameters or cross-platform/cross-runtime portability. Mitigating this remains future work, while ambitious task T4 and stretch task T5 explore resilience and write safety within this architecture.
- **Client placement.** k6 runs in the same region, so results describe server-side behavior and exclude the Internet latency that distant users experience.
- **Opaque scheduling.** Lambda decides when to reuse or create environments; I observe this through cold-start and environment-ID headers but cannot control it.

5 Testing and Evaluation Plan

5.1 Correctness Tests

Table 5 lists the planned tests. Unit tests use a fake clock and a stub dependency, so timing rules are checked without real waiting; integration tests run against the deployed stack. All are automated (pytest, run by `make test`).

Table 5: Planned correctness tests.

Test / component	Setup or input	Expected outcome	How checked
CT1. Normal workflow (M1–M4)	Deployed stack, Base, no faults; `GET /products/42`, then `POST /orders` (quantity 2).	Both 200; correct product returned; stock decreases by exactly 2; diagnostic headers present; `X-Cold-Start` true only on an environment's first request.	Integration test reads DynamoDB before and after.
CT2. Invalid input (M1)	IDs `0`, `1001`, `abc`; quantity 0 and 11; unknown ID.	400 for malformed input, 404 for unknown ID; the dependency client is never called.	Unit test with a stub that records calls.
CT3. Retry limits (M2)	Stub returns 503, 503, 200; always 503; 400; a call that never returns.	Succeeds on attempt 3; stops after 4 attempts; never retries a 400; no attempt starts with under 50 ms left, and total time stays under 3,000 ms; every sleep lies in $[0, \min(1000, 100 \cdot 2^n)]$ ms.	Unit test with fake clock and seeded jitter.
CT4. Cache TTL (M2)	Same `GET` at $t = 0$, 59, and 61 s; one `POST`.	Miss at 0 s, hit at 59 s (no dependency call), miss at 61 s; `POST` is never cached.	Unit test with fake clock.
CT5. Fault injector (M4)	10,000 calls at rate 10%; delay +300 ms; outage from 60 to 120 s.	9–11% injected 503s; median latency +270–330 ms; 100% failures inside the window and none outside.	Calibration script against the deployed dependency.
CT6. Metrics (M7)	Hand-built fixture of 20 requests with known latencies, statuses, billed durations, and one outage.	$p_{50}/p_{95}/p_{99}$ (nearest rank), error rate, recovery time R, and cost C equal the values I computed by hand.	Unit test, exact match.
CT7. Failed collection (M6)	k6 run forced to drop iterations; S3 upload made to fail.	Run is marked invalid, re-queued, and excluded from analysis; no partial data appear in summaries.	Integration test of the orchestrator.

5.2 Evaluation

The metrics, comparison groups, workloads, and repetitions are defined in Section 4: each metric maps to one sub-question (latency percentiles and error rate to RQ1, hit ratio and cold starts to RQ2, cost C and staleness to RQ3), and every mechanism is compared against Base under the same condition and session. Two controls help separate competing explanations. First, Base is run twice in every session (an A/A comparison); differences smaller than this noise floor are not reported as effects. Second, the diagnostic headers let each result be attributed to a cause: (i) if Retry lowers the error rate in C4, the reduction should be explained by requests with `X-Attempts` > 1 that matched injected faults in the fault log, not by a quieter period; (ii) if caching helps less in C3 at high rates, per-environment hit ratios (grouped by `X-Env-Id`) separate empty caches in new environments (H2) from TTL expiry or the Zipf tail, which would also lower hit ratios in long-lived environments; (iii) a latency spike during

the ramp is attributed to cold starts (`Init Duration` in `REPORT` lines), throttling (429 responses), or load-generator saturation (dropped iterations, EC2 CPU).

5.3 Reproducibility

All code, infrastructure, and configuration live in one Git repository. Terraform defines every AWS resource, with the provider version pinned by the lock file; Python dependencies are pinned in `requirements.txt`; k6 runs from a pinned Docker image tag; the EC2 load host uses a fixed AMI and instance type. A single experiment plan file lists configurations, conditions, repetitions, and the random seed that fixes the catalog, request sequence, run order, and injected faults. `make deploy`, `make experiment`, `make analyze`, and `make destroy` reproduce the study, and the README documents the AWS region and the required concurrency quota. Each run's raw k6 output, `REPORT` lines, fault log, and a manifest (Git commit, configuration, timestamps) are stored in a versioned S3 bucket and exported with the final submission, so every table and figure can be regenerated from saved data without re-running the experiments.

6 Critical Design Decision and Trade-off Analysis

Decision: where to cache product reads.

1. Alternatives. (A) An *in-function cache*: a dictionary in each Lambda execution environment (M2), behind the cheaper HTTP API. (B) An *API Gateway response cache*: a 0.5 GB stage cache with a 60 s TTL keyed by the product path, which requires switching to the REST API type. In B, a hit is answered by the gateway and never invokes Lambda.

2. Choice: A. RQ2 asks how the serverless instance lifecycle affects application-level caching, so the cache must live inside the execution environment being studied; B would hide exactly that behavior. A also has no fixed fee and keeps the HTTP API at $1.00 per million requests instead of $3.50, which matters within a $119 credit, and its hits remain visible per environment through `X-Cache` and `X-Env-Id`.

3. Trade-off. A's main drawback is fragmentation: every new environment starts empty, so the hit ratio may fall during bursts, exactly when protection is needed, and every request, including a hit, still invokes Lambda and uses concurrency. B shares one cache across all environments and absorbs bursts before Lambda. On cost, B never pays off here: the REST API adds $2.50 per million requests, while B's extra saving over A is only the short Lambda invocation that A spends serving a hit (about $0.22 per million hits at 256 MB and 5 ms), plus B's fixed fee of about $0.48 per day. B is therefore preferable only when its latency and burst protection are worth that premium, for example a read-heavy API with large bursts where cold starts break the SLO.

4. Challenge to the reasoning. Condition C3 tests this directly. If, at 1,000–2,000 req/s, `Cache`'s aggregate hit ratio falls below 50% and its p_{95} exceeds 500 ms in windows where `Init Duration` shows cold starts, the per-instance design is failing for the reason B fixes. I would then run C3 once against a REST API stage with B enabled; if B keeps $p_{95} \leq 500$ ms at the same rates, B is the better choice for bursty workloads despite its cost.

AI suggestion revised. An early draft hypothesis, written with AI assistance, claimed the gateway cache served requests "at the edge" and broke even at about 40,000 requests per day. Checking AWS documentation showed the cache is regional and available only for REST APIs. A later AI-produced estimate (about 2 million requests per day) compared B with no caching and ignored the REST API's higher request price; re-deriving it against A, as above, showed B does not break even on cost at any volume. I kept the corrected analysis.

7 Project Schedule, Risks, and Development Practices

7.1 Schedule

Period	Concrete outputs	Completion check	Depends on
Oct 21–Nov 3	T1: Terraform stack, catalog seeder, M1, M3, M4; CT1, CT2, CT5.	One-command deploy and teardown work; CT1, CT2, and CT5 pass.	—
Nov 4–17	T2: retry and cache (M2); T3 pipeline (M5–M7); CT3, CT4, CT6, CT7; pilot of C1 and C4 for all four configurations.	Pilot data flow end to end into a summary table; all unit tests pass.	Nov 3
Nov 18–Dec 1	Full 140-run grid; analysis tables and figures; T4 (breaker) only if the grid finishes by Nov 24. Buffer for the Thanksgiving week.	All runs valid and stored in S3; `make analyze` regenerates every figure.	Nov 17
Dec 2–10	T5 if time permits; final report; README and reproducibility check; stack teardown; submission.	A fresh clone redeploys and reproduces one cell within its reported range; report submitted Dec 10.	Dec 1

Table 6: Biweekly milestones.

7.2 Risks

The original top risk, a Lambda concurrency quota of 10, was resolved when AWS raised it to 1,000 on October 15. **Risk 1: cloud cost exceeds the $119 credit** (for example, a runaway load test), which would stop the experiments. Prevention: a $20 budget alarm, reserved concurrency of 400 per function, fixed k6 durations, and teardown after every session; the full

grid is estimated at $15–25. *Fallback:* reduce repetitions from five to three and cap the ramp at 1,000 req/s, which keeps all four configurations and seven conditions and therefore the essential comparison. **Risk 2: implementation takes longer than planned**, especially the measurement pipeline (M6–M7), leaving too little time for the full grid. *Fallback:* drop the optional T4 and T5, compute metrics from k6 output and `REPORT` lines only (omitting the staleness metric), and run three repetitions; every B10 condition and both essential mechanisms are still evaluated.

7.3 AI-Agent Plan and Disclosure

I used Claude (Anthropic) and Gemini (Google) while preparing this document: to interpret the assignment, explain concepts, structure sections, check AWS pricing, and draft LaTeX text and the Mermaid diagrams. I made and can defend each design decision. One corrected suggestion is the cache break-even estimate described in Section 6. **Delegated development task.** I plan to delegate one bounded task to an agent: implementing the retry policy in M2 (`reliability.py` only) to the rules in Table 2. Before delegating, I will write tests CT3 and CT4 myself from Section 5, so the agent's code is checked against independent expectations rather than its own tests. I will then review the code line by line, run the tests, and sample 10,000 backoff delays to confirm they stay within the jitter bounds.

References

[1] M. Brooker, "Exponential Backoff and Jitter," *AWS Architecture Blog*, 2015. `https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/`

[2] M. Brooker, "Timeouts, retries, and backoff with jitter," *Amazon Builders' Library*, 2019. `https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/`

[3] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, "Peeking Behind the Curtains of Serverless Platforms," in *Proc. USENIX ATC*, 2018.

[4] M. Shahrad et al., "Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider," in *Proc. USENIX ATC*, 2020.